

Call by Value and Call by Reference in C# with Examples

In this article, I am going to discuss **Call By Value and Call By Reference** in C# with Examples. Please read our previous article discussing [User-Defined Functions in C#](#) Language with Examples.

Call by Value and Call by Reference in C#:

It is a common question that is asked by interviewers in interviews “What is Call/Pass by Value, and What is Call/Pass by Reference”. Most beginner- or intermediate-level developers have misconceptions about it and answer it incorrectly during interviews. They generally say when we pass primitive types, they are passed by value, and when we pass reference types, they are passed by reference. This answer is wrong. So, today I will explain to you the real difference between them.

Call By Value in C#

In .NET Framework, by default, all the objects are called by value, not called by reference. So, whether it is a Value Type (Primitive data types like int, char, double, etc.) or Reference Data Type (class, interface, delegate, string, etc.), they will be called by value by default. Let us understand Call by Value in C# with some examples. Consider the following Example. In the following example, I am using the value data type.

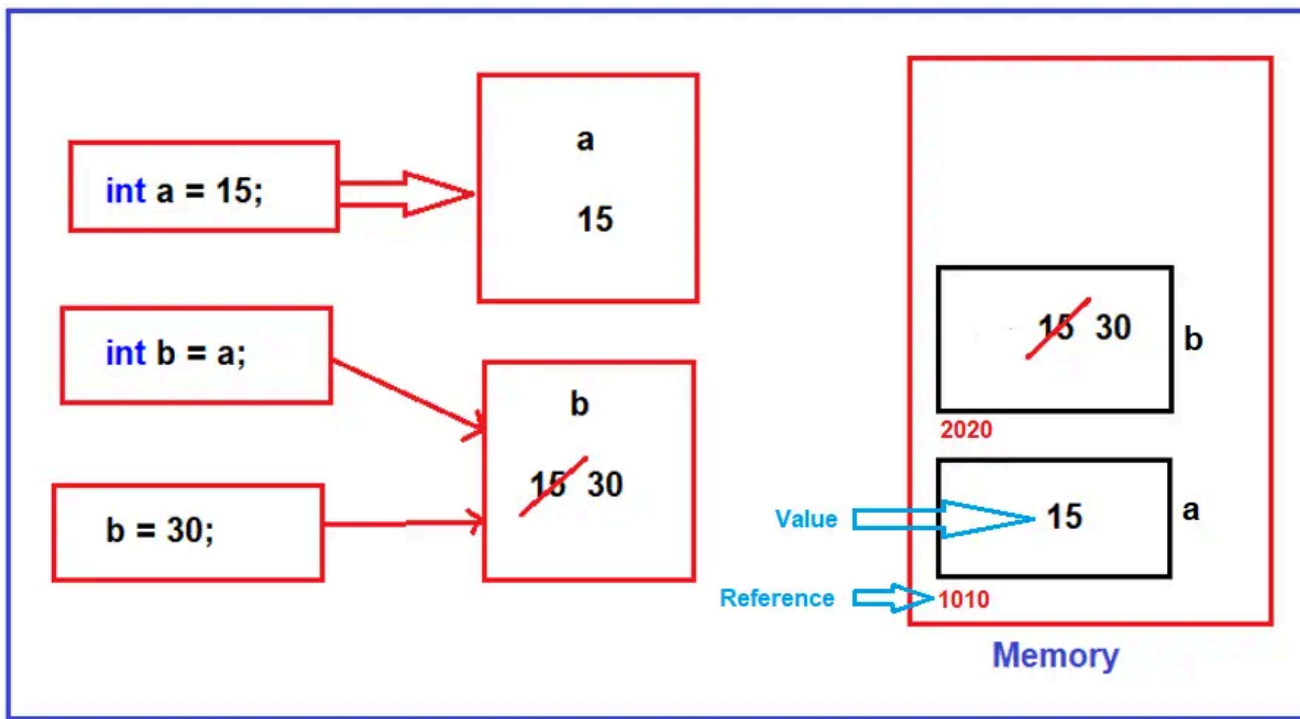
```
using System;
namespace FunctionsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 15;
            int b = a;
            b = 30;
            Console.WriteLine(a);
            Console.ReadKey();
        }
    }
}
```

So, what will be printed on the Console?

If your answer is 15, then you are absolutely right because **int** is a value data type, and by default, it is passed by value, which means for the above code, the variable “a” has stored the value 15 in it. When we create the variable b and assign it a, the value of a is copied to b, and after that, if we change b, it will not affect a. This is because we have copied the value of a to b.

Memory Representation of Call by Value in Value Types in C#:

For a better understanding, please have a look at the below image. Let us assume that the memory reference of variable a is 1010, and it holds the value 15. And when we execute the statement **int b = a;** in this case, it will pass the value i.e, 15 to the variable b. This is called by value mechanism in C# with value or primitive data type.



The same thing is also happening when we pass value types to methods. For a better understanding, please have a look at the below example. Here, we have created a method called `UpdateValue` that updates the value of an `int` variable `b` to `30`.

```
using System;
namespace FunctionsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 15;
            UpdateValue(a);
            Console.WriteLine(a);
            Console.ReadKey();
        }
        private static void UpdateValue(int b)
        {
            b = 30;
        }
    }
}
```

So, what will be printed on Console now?

Yes, it will print `15` in the console. As I already said, that value is copied, so when the `UpdateValue` method is called, variable `a` value is copied to variable `b`, so changing variable `b` does not change variable `a`.

Call by Value with Reference Types in C#

I have created the following Employee class to understand Call by Value with Reference Types in C#. We already know that class is a reference data type.

```
public class Employee
{
    public int EmployeeID;
    public string Name;
}
```

Now, please observe the following code. Here, we create an instance of the Employee class, i.e., Emp1, and set its two data members. Next, we assign it (Emp1 instance) to another instance of the Employee class, i.e., Emp2, and change the Name property, and then we print EmployeeID and Name on the Console to check what is printed:

```
using System;
namespace FunctionsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee Emp1 = new Employee();
            Emp1.EmployeeID = 1001;
            Emp1.Name = "James";
            Employee Emp2 = Emp1;
            Emp1.Name = "Smith";
            Console.WriteLine($"Emp1 Name = {Emp1.Name}");
            Console.WriteLine($"Emp2 Name = {Emp2.Name}");
            Console.ReadKey();
        }
    }
    public class Employee
    {
        public int EmployeeID;
        public string Name;
    }
}
```

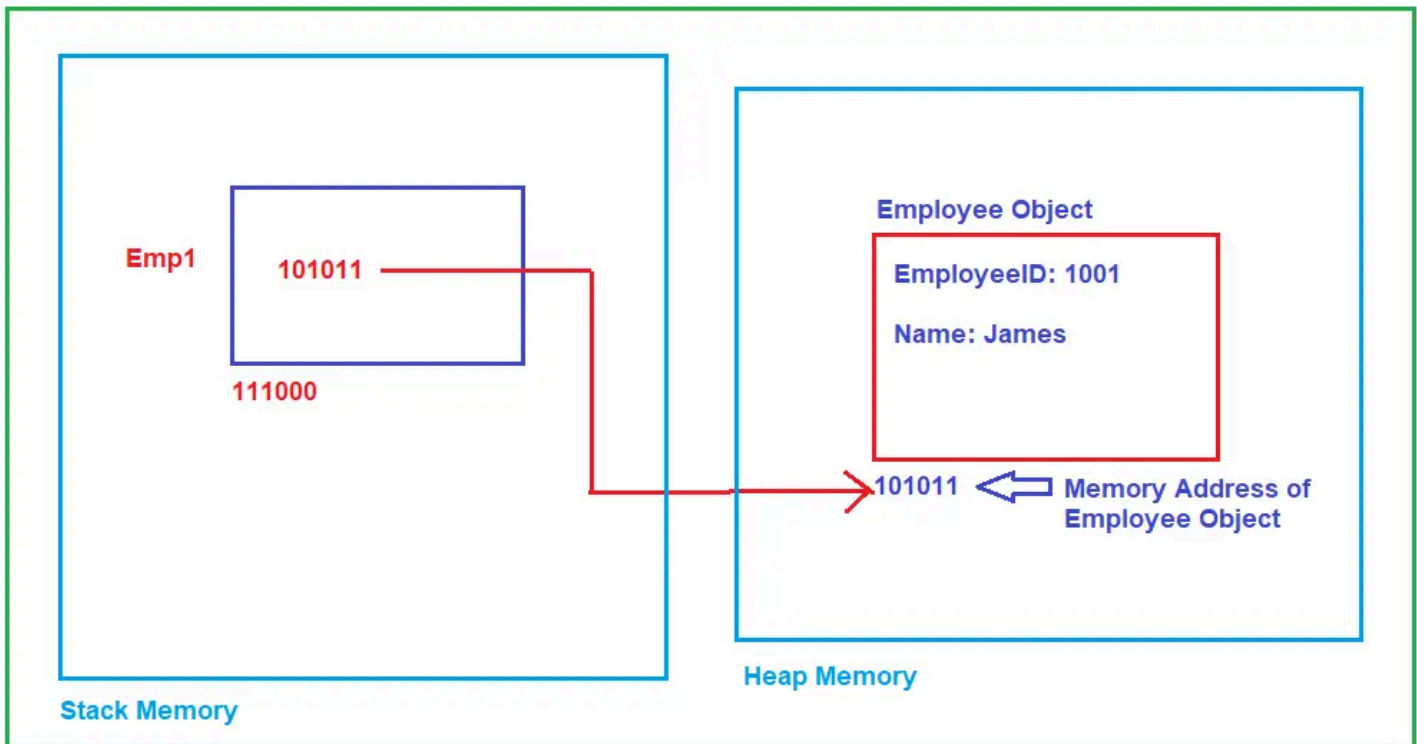
When we create an Instance of the Employee class, an object is created inside the heap memory, and memory is allocated to it. We are storing the reference of that memory location in the Emp1 reference variable, mostly in stack memory.

When we assign Emp1 to Emp2, the reference of the object memory location that Emp1 is holding is copied to Emp2. Now we have two copies of reference, but they both point to the same memory location. So, changing the Name property's value will change the object's value in memory, which we have references in Emp1 and Emp2. So, "Smith" will be printed on the console for both references. So, when you run the above code, you will get the following output.

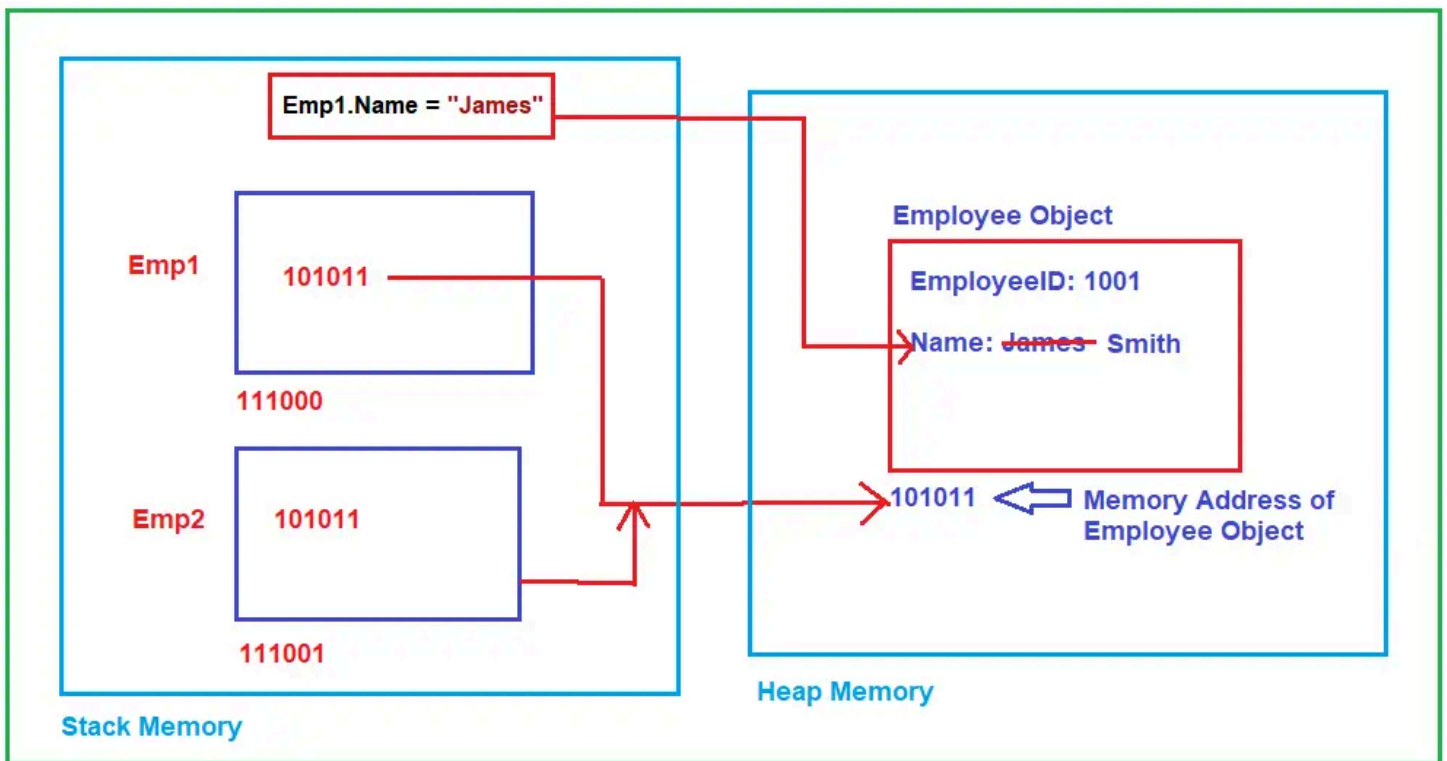
```
Emp1 Name = Smith
Emp2 Name = Smith
```

Memory Representation Call by Value with Reference Types in C#

When we create the first instance of the Employee object, i.e., Emp1, then we need to understand what happens inside the memory. In heap memory, one object will be created, which will allocate the memory for EmployeeID and Name data members. Then, let's assume that the object's memory address is 101011, and then that memory reference will be stored inside the Emp1 reference variable. Again, assume that the memory address of the Emp1 reference variable is 111000. So, the point that you need to remember is now Emp1 is a reference variable whose address is 111000, and it holds the reference address 101011.



Now, let us understand when we assigned the Emp1 reference variable to the Emp2 reference variable, i.e., **Employee Emp2 = Emp1**; and what happens when we update the Name on the Emp1 reference variable, i.e., **Emp1.Name = "Smith"**; For a better understanding, please have a look at the following image. When **Employee Emp2 = Emp1**; statement is executed, what value Emp1 is holding that will be copied to the Emp2 reference variable, Emp1 is holding 101011, so that will be copied to Emp2 as well. This is called by value with a reference type. Because here, we are not copying the memory address of Emp1, which is 111000; instead, we are copying the value which is held by Emp1, i.e., 101011, which is nothing but the memory address of the Employee object, which is created inside the Heap Memory. That means both reference variables are now pointing to the same memory location. Now, when **Emp1.Name = "Smith"**; statement is executed, it will update the Name to Smith of the Employee object in the heap memory, and hence the changes will be reflected in the reference variables.



We can also get the same behavior using a method. For a better understanding, please have a look at the following example.

```
using System;
namespace FunctionsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee Emp1 = new Employee();
            Emp1.EmployeeID = 1001;
            Emp1.Name = "James";
            UpdateName(Emp1);
            Console.WriteLine($"Emp1 Name = {Emp1.Name}");
            Console.ReadKey();
        }
        public static void UpdateName(Employee Emp2)
        {
            Emp2.Name = "Smith";
            Console.WriteLine($"Emp2 Name = {Emp2.Name}");
        }
    }
    public class Employee
    {
        public int EmployeeID;
        public string Name;
    }
}
```

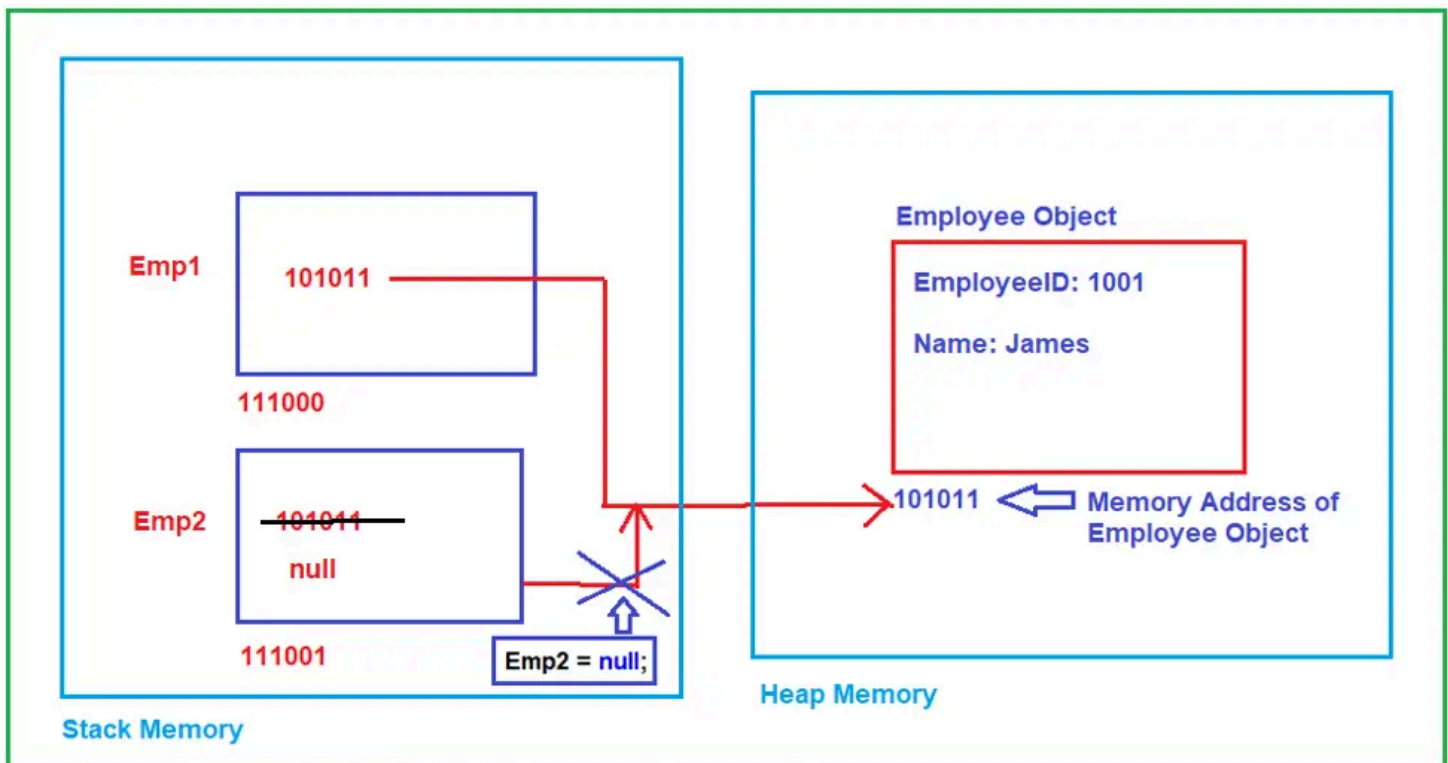
When we pass reference Emp1 of the Employee object to the UpdateName method, the memory location reference is copied to the local object Emp2 of the method UpdateName. And both (Emp1 and Emp2) are pointing to the same memory location. So, changing the Name variable's value will change the object's value in memory, and the changes will reflect in both the references, i.e., Emp1 and Emp2, and the updated value Smith will print on the console.

What would happen if we set the Emp2 to null inside the UpdateName method as follows?

```
using System;
namespace FunctionsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee Emp1 = new Employee();
            Emp1.EmployeeID = 1001;
            Emp1.Name = "James";
            UpdateName(Emp1);
            Console.WriteLine($"Emp1 Name = {Emp1.Name}");
            Console.ReadKey();
        }
        public static void UpdateName(Employee Emp2)
        {
            Emp2 = null;
        }
    }
    public class Employee
    {
        public int EmployeeID;
        public string Name;
    }
}
```

Now, if you think that you will get a Null Reference Exception in the Main method in the WriteLine statement where you print the Name, then you are wrong. If you think that it will output James, then you are right, and you have understood that by default, reference is passed by value in C#, not by reference.

Memory Representation of the above Example:



As you can see in the above image, when we set **Emp2 = null;** then the Emp2 references are deleted. But still, the Emp1 reference to the Employee object exists. This is the Call by Value mechanism in C#. Now, I hope you understand the Call by Value Mechanism in C# using value and reference types with Examples. If we want to make Emp1 also null, when you set Emp2 as null, then you have to pass it to the method via reference, which is done in C# by using the ref Keyword, i.e., nothing but pass by reference in C#. Let us understand Call by Reference in detail.

Call By Reference in C#

Let us understand Call by Reference or Pass by Reference with both value types as well as with reference type in C# with examples. We need to prefix the parameter with the ref keyword for the call by reference. I am going to explain Call by Reference using the same examples we discussed in Call by Value so that it will be easier for you to understand the concept and the differences between them.

Call by Reference with Value Types in C#

We will use the same example, but this time, we will be calling by reference. So, what we need to do is, we need to prefix the variable with the ref keyword. Please have a look at the following example. Here, you can see, inside the UpdateValue method, instead of **int b**, we are now using **ref int b**. Here, we have added the ref keyword with the input parameter, meaning the argument must be passed by reference when we call this method.

```
using System;
namespace FunctionsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

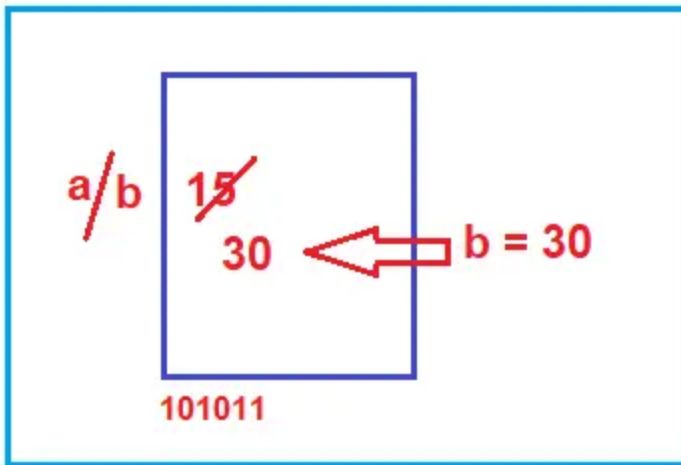
    int a = 15;
    UpdateValue(ref a);
    Console.WriteLine(a);
    Console.ReadKey();
}
static void UpdateValue(ref int b)
{
    b = 30;
}
}

```

Then we call the UpdateValue method, but we have to use the ref keyword before the argument name. This is Call by Reference in C# with Value Type. If you do not use the ref keyword, you will get a compile-time error.

When you run the above code, it will print 30 on the console. This is because of the ref keyword. In the above example, when the UpdateValue method is called, the incoming parameter b has the same memory address as a, which is passed as the argument. That's why modifying the value of b would also reflect the change in a. In this case, a new memory location is not created for method parameter b. Here, you can say b just as an alias of a or an alternative name of a. So, if we make any changes using variable b, it will also reflect in variable a.

For a better understanding of the above example, please look at the diagram below, which shows the memory representation of the above example.



Call by Reference with Reference Types in C#

Now, we will see the same thing with a reference type. The behavior is going to be the same for reference types as well. We are also going to work with the same example that we worked with Call by Value with Reference Type. So, please have a look at the following example. Now, in the UpdateName method, we have added the ref keyword. In the main method, while calling the UpdateName method, we also prefixed the variable Emp1 with the ref keyword as follows:

```

using System;
namespace FunctionsDemo
{

```

```

class Program
{
    static void Main(string[] args)
    {
        Employee Emp1 = new Employee();
        Emp1.EmployeeID = 1001;
        Emp1.Name = "James";
        UpdateName(ref Emp1);
        Console.WriteLine($"Emp1 Name = {Emp1.Name}");
        Console.ReadKey();
    }
    public static void UpdateName(ref Employee Emp2)
    {
        Emp2 = null;
    }
}
public class Employee
{
    public int EmployeeID;
    public string Name;
}
}

```

Now, when the UpdateName method sets the Emp2 to null, this also sets the Emp1 to null. This is because of Call By Reference. So, in this case, the reference of Emp1 is passed to the method, and hence a new reference is not created for Emp2. So, when you run the above code, you will get a runtime error as shown in the below image. So, here, you can consider Emp2 as an alias of Emp1 or an alternative name for Emp1.

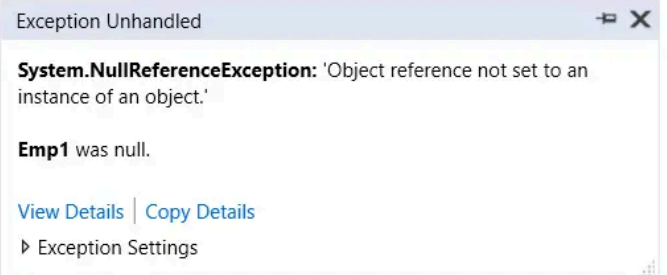
```

class Program
{
    static void Main(string[] args)
    {
        Employee Emp1 = new Employee()
        {
            EmployeeID = 1001,
            Name = "James"
        };
        UpdateName(ref Emp1);
        Console.WriteLine($"Emp1 Name = {Emp1.Name}");
        Console.ReadKey();
    }

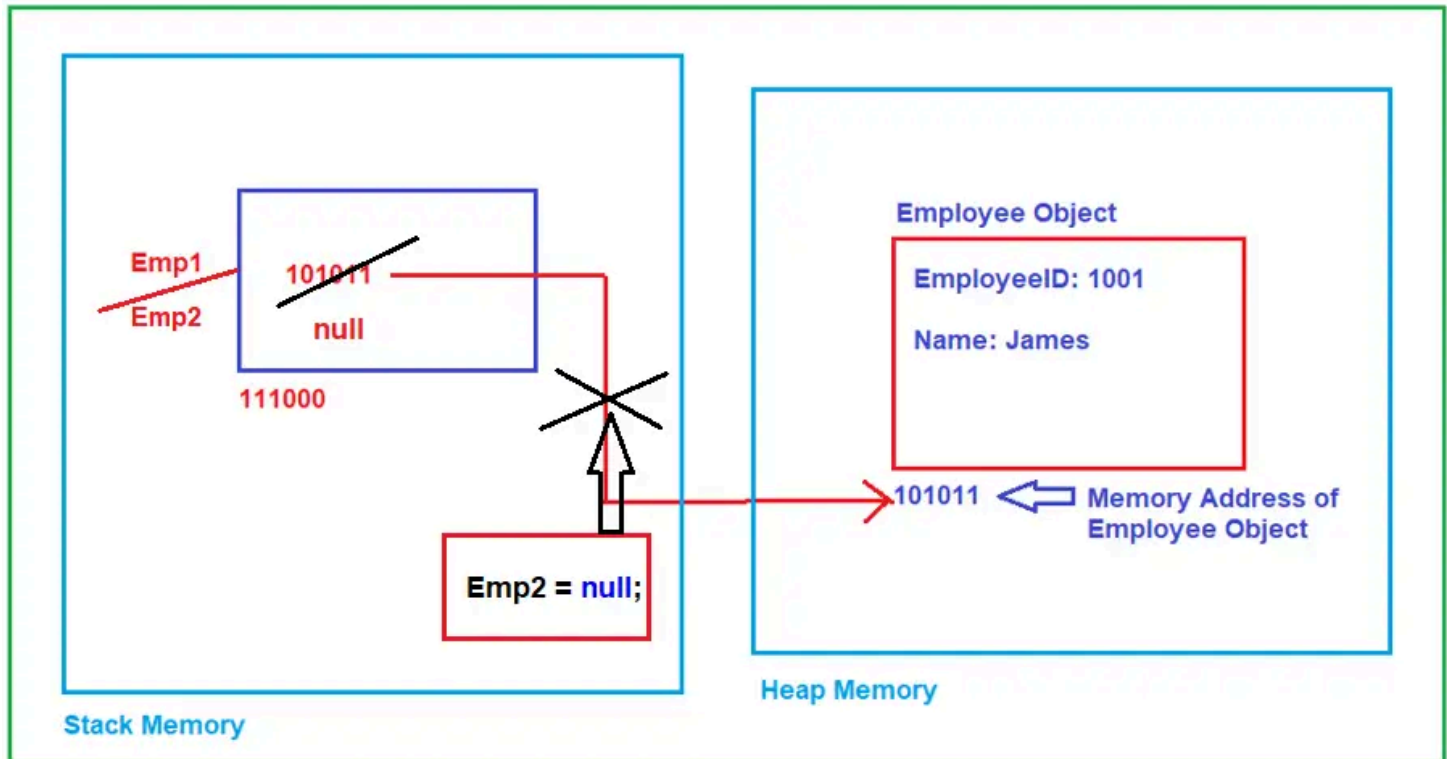
    public static void UpdateName(ref Employee Emp)
    {
        Emp = null;
    }
}

public class Employee
{
    public int EmployeeID { get; set; }
    public string Name { get; set; }
}

```



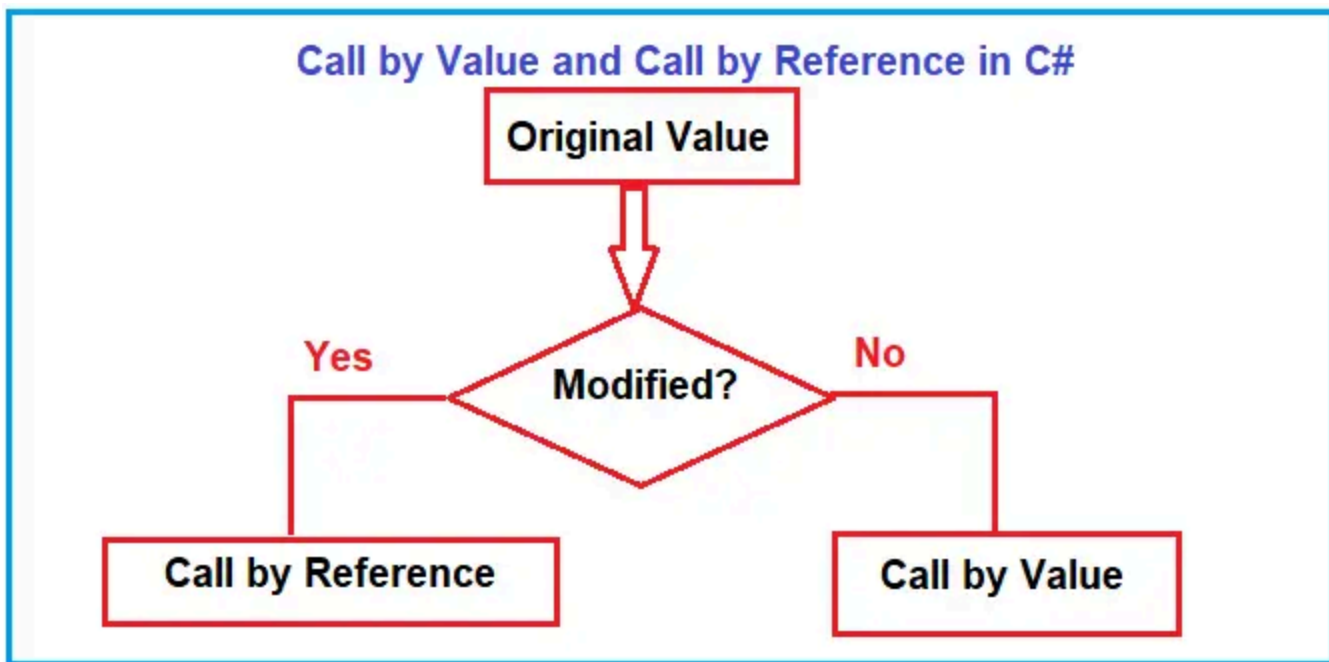
To understand why we are getting the Null Reference Exception, we need to understand the memory representation of the above example. Let us assume the memory address of Emp1 is 111000, and it is pointing to the Employee object whose address is 101011. That means the Emp1 address is 111000, and it is holding the Employee Object reference address 101011. In the case of Call by Reference, when we pass ref Emp1, then, in that case, we are passing the memory address of Emp2, i.e., 111000. And on the computer, we cannot have the same memory address twice. So, here Emp2 is also pointing to the same memory address as Emp1. We can say that Emp2 is an alias of Emp1 or an alternative name of Emp1. So, when we make Emp2 null, that means the memory address 111000, which is pointing to the Employee object, i.e., 101011, will become null, and then we try to access the Name property, we get the runtime exception.



Note: If you want to achieve Call by Reference in C#, you must have to use either the out or ref keyword, regardless of whether the argument type is a value type or a reference type. In our upcoming article, we will discuss the need and use of ref and out keywords in detail.

Difference between Call by Value and Call by Reference in C#:

The most important point you need to remember is that in C#, all objects/values are called by value by default, not by reference, regardless of whether they're Value Types or Reference Types. With this kept in mind, let us proceed and see the differences.



Call By Value in C#:

1. In Call by Value, the copy of the original variable is passed to the called function.
2. In Call by Value, the value of the original parameter is copied into the parameter of the function. As a result, if we make any modifications to formal parameters, they don't have any impact on the actual parameter.
3. It will not allow you to change the actual variables using function calls.
4. The memory location referred to by formal parameters and actual arguments is different.
5. It doesn't require a ref or out keyword in C#.

Call By Reference in C#:

1. In Call by Reference, the variable's reference (or the actual address) is passed to the called function.
2. In Call by Reference, the formal and actual parameters point to the same memory address. As a result, any changes made to the formal parameters are also reflected in the actual parameters.
3. It allows you to change the actual variable's value using function calls.
4. The memory location referred to by formal parameters and actual arguments are the same.
5. It requires a ref or out keyword to achieve Call By Reference in C#.

In the next article, I am going to discuss [Recursion in C#](#) with Examples. In this article, I try to explain **Call By Value and Call By Reference in C#** with Examples. I hope you enjoy this Call by Value and Call by Reference in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.